AUTOLUX

Memória Descritiva

Projeto de gestão de um armazém de peças automóveis

Índice

1. Introdução
2. Descrição do produto
3. Arquitetura e opções técnicas
4. Ferramentas utilizadas
5. Processo de desenvolvimento e decisões técnicas
6. Testes realizados e resultados
7. Dificuldades, limitações e melhorias futuras
8. Conclusão

1. Introdução

Este documento apresenta a memória descritiva do projeto AutoLux, uma aplicação web destinada à gestão de um armazém de peças automóveis. O documento descreve o objetivo do produto, as opções técnicas adotadas, as ferramentas utilizadas, as decisões tomadas durante o desenvolvimento, os testes efetuados e as funcionalidades que poderão ser acrescentadas futuramente.

A aplicação foi desenvolvida com uma arquitetura dividida entre uma interface PHP, uma API em Node.js e duas bases de dados MySQL. O desenvolvimento foi realizado por etapas, testando as ligações e as funcionalidades principais à medida que foram sendo implementadas.

2. Descrição do produto

2.1. Destinatários e objetivo

A AutoLux destina-se principalmente aos funcionários de um armazém de peças automóveis que necessitam de consultar o catálogo e o stock, registar vendas a clientes e gerir encomendas a fornecedores. O objetivo é disponibilizar estas operações numa interface web, centralizando a consulta e o registo de informação.

2.2. Funcionalidades implementadas

O catálogo de peças apresenta informação como referência, nome, marca, tipo, preço e stock disponível. Inclui filtros por marca, tipo de peça e intervalo de preços, para facilitar a pesquisa.

A página de vendas permite selecionar um cliente, escolher um método de pagamento e indicar as peças e quantidades a vender. O servidor valida os dados, consulta os preços guardados na base de dados, verifica a disponibilidade do stock, calcula o total e regista a venda. Após uma venda válida, o stock é atualizado.

A componente de fornecedores permite consultar fornecedores e encomendas, bem como criar novas encomendas através da interface PHP. A interface comunica com a API Node.js, que valida e processa os pedidos e guarda os dados na segunda base de dados MySQL.

2.3. Organização da interface

A aplicação inclui uma página inicial com ligações para as principais funcionalidades, permitindo navegar entre o catálogo, as vendas, a consulta de fornecedores, a consulta de encomendas e o formulário de criação de encomendas.

3. Arquitetura e opções técnicas

3.1. Visão geral

A solução está dividida em três componentes principais: interface e lógica de vendas em PHP; API REST de fornecedores e encomendas em Node.js; e armazenamento em duas bases de dados MySQL. Esta separação permite manter as operações de vendas e as operações de compras a fornecedores em componentes distintos.

3.2. PHP e primeira base de dados

A aplicação PHP liga-se diretamente à base de dados autolux_vendas através de PDO. Esta base de dados contém as tabelas relacionadas com clientes, peças, vendas e linhas de venda. O PHP é

responsável pela apresentação das páginas, pelo processamento dos formulários e pela lógica de negócio das vendas.

A ligação à base de dados foi colocada no ficheiro config/database.php, separando a configuração da ligação do código das páginas. Antes de desenvolver as funcionalidades dependentes da base de dados, foi criada uma página temporária de teste para confirmar que a ligação funcionava e que os registos de peças podiam ser consultados.

3.3. Node.js, Express e segunda base de dados

A API foi desenvolvida em Node.js com Express. A configuração da ligação à base de dados foi separada no ficheiro database.js, enquanto o ficheiro server.js contém as rotas e o processamento dos pedidos. A API utiliza mysql2/promise para comunicar com a base de dados autolux_fornecedores.

A API disponibiliza endpoints para consultar, criar, atualizar e desativar fornecedores, bem como consultar, criar e atualizar encomendas. Os dados são trocados em JSON através de pedidos HTTP. A interface PHP não consulta diretamente a base de dados dos fornecedores; utiliza os endpoints da API, mantendo a separação de responsabilidades.

3.4. Bases de dados

A base de dados autolux_vendas armazena os dados relativos aos clientes, às peças, às vendas e às respetivas linhas de venda. A base de dados autolux_fornecedores armazena os fornecedores, as encomendas a fornecedores e as peças associadas a cada encomenda.

Para facilitar a instalação e a preparação do ambiente de desenvolvimento, foram criados scripts SQL separados nos dois repositórios do projeto. No repositório PHP, o ficheiro sql/01_criar_base_dados.sql permite criar a estrutura da base de dados de vendas e o ficheiro sql/03_inserir_dados_demo_vendas.sql insere dados de demonstração. No repositório Node.js, o ficheiro sql/02_criar_base_dados_fornecedores.sql cria a estrutura da base de dados de fornecedores e o ficheiro sql/04_inserir_dados_demo_fornecedores.sql insere os respetivos dados de demonstração.

Os dados de demonstração permitem testar as funcionalidades da aplicação sem introduzir manualmente todos os registos. Os nomes, contactos, preços e restantes valores utilizados para este efeito são fictícios e não representam informação comercial real.

3.5. Validação, segurança e consistência

Nas consultas SQL foram utilizadas consultas preparadas, reduzindo os riscos associados à inclusão direta de valores fornecidos pelo utilizador. Na criação de encomendas, a API valida o fornecedor, a existência de peças e os respetivos campos, e calcula o total no servidor.

No registo de vendas e na criação de encomendas, foram utilizadas transações MySQL para agrupar operações relacionadas. Assim, o registo principal e as respetivas linhas são tratados como uma única operação, reduzindo o risco de ficarem dados incompletos em caso de erro. A venda também verifica o stock disponível antes de atualizar as quantidades.

4. Ferramentas utilizadas

- **PHP:** Desenvolvimento das páginas dinâmicas e processamento das vendas.

- **Node.js:** Execução da API responsável pela gestão de fornecedores e encomendas.
- **Express:** Definição das rotas HTTP da API REST.
- **MySQL:** Armazenamento relacional dos dados da aplicação.
- **PDO:** Ligação do PHP à base de dados autolux_vendas.
- **mysql2/promise:** Ligação do Node.js à base de dados autolux_fornecedores.
- **XAMPP:** Execução local do Apache e MySQL no ambiente de desenvolvimento.
- **phpMyAdmin:** Consulta e administração das bases de dados e verificação dos registos durante os testes.
- **npm:** Instalação das dependências e execução dos scripts do projeto Node.js.
- **Visual Studio Code:** Edição do código e execução de testes HTTP através do terminal.
- **Git e GitHub:** Controlo de versões, histórico de alterações e publicação dos dois repositórios.
- **Navegador web:** Acesso à interface PHP e realização de testes funcionais.

5. Processo de desenvolvimento e decisões técnicas

5.1. Preparação do ambiente e testes iniciais

O desenvolvimento começou pela preparação do ambiente local com XAMPP, PHP, MySQL, Node.js, npm, Git e Visual Studio Code. Foram criadas as duas bases de dados e inseridos dados de teste para permitir validar as consultas e as funcionalidades sem depender de introduções manuais repetidas.

Uma das dúvidas iniciais foi decidir se deveria ser desenvolvida primeiro a interface PHP ou a API Node.js. O projeto acabou por avançar por componentes, validando primeiro as ligações e as rotas da API e desenvolvendo depois as páginas PHP que consomem esses endpoints. Esta abordagem permitiu testar cada parte antes de integrar o conjunto.

5.2. Desenvolvimento progressivo da API

A API foi construída gradualmente. Primeiro, foi separada a configuração da ligação à base de dados no ficheiro database.js. Em seguida, foram implementadas rotas para consultar fornecedores, consultar um fornecedor por identificador, criar fornecedores, atualizar os seus dados e desativá-los. A desativação altera o campo ativo para 0 em vez de apagar definitivamente o registo, preservando o histórico.

Posteriormente, foram criadas rotas para consultar a lista de encomendas e obter os detalhes de uma encomenda específica. A listagem utiliza um INNER JOIN para apresentar o nome do fornecedor associado. A consulta individual obtém os dados gerais da encomenda e as peças que a compõem.

Quando se verificou que faltavam linhas de peças para algumas encomendas de demonstração, foram preparados dados fictícios para a tabela encomenda_pecas. Os totais foram recalculados com base nas quantidades e preços unitários, para que os detalhes das encomendas fossem coerentes com o valor total registado.

A rota de criação de encomendas valida os dados recebidos, verifica se o fornecedor existe e está ativo e calcula o total no servidor. A encomenda e as respetivas linhas são inseridas através de uma transação. Foi também implementada a atualização do estado da encomenda, permitindo apenas os estados Pendente, Enviada e Recebida.

5.3. Desenvolvimento do catálogo

O catálogo foi desenvolvido em catalogo.php e ligado à base de dados através da configuração PDO. Foram implementados filtros por marca, tipo e intervalo de preços. As opções de marca e tipo são

obtidas da própria base de dados, evitando manter listas fixas no código. Para executar as pesquisas, foram utilizadas consultas SQL preparadas.

5.4. Desenvolvimento das vendas e atualização do stock

A página vendas.php permite selecionar um cliente, escolher o método de pagamento e indicar as peças e quantidades. Os preços são consultados na base de dados e o total é calculado no servidor, evitando depender de valores enviados pelo formulário. O sistema verifica a existência do cliente e a disponibilidade do stock antes de registar a venda.

A operação de venda utiliza uma transação para registar a venda, inserir as respetivas linhas e atualizar o stock. A validação e atualização do stock são feitas no servidor; a utilização de SELECT ... FOR UPDATE permite bloquear as linhas relevantes durante o processamento da transação e reduzir conflitos em operações simultâneas.

5.5. Integração da interface PHP com a API

Depois de validar as rotas da API, foram desenvolvidas as páginas fornecedores.php e encomendas.php para apresentar os dados recebidos através de pedidos GET. Foi também desenvolvida encomendas_nova.php, que obtém a lista de fornecedores e envia um pedido POST com os dados da nova encomenda em JSON.

Esta decisão permite que a interface PHP apresente os dados e recolha os dados introduzidos pelo utilizador, enquanto a API Node.js mantém a responsabilidade pelo processamento e persistência dos fornecedores e encomendas. Os testes confirmaram que a encomenda criada pela interface aparece posteriormente na listagem.

5.6. Controlo de versões

Foram utilizados repositórios Git separados para a interface PHP e para a API Node.js. As alterações foram registadas e enviadas para o GitHub, permitindo acompanhar a evolução do código. No projeto Node.js foi criado o ficheiro .env.example com as variáveis necessárias à configuração, enquanto o ficheiro .env está excluído pelo .gitignore para evitar publicar configurações locais.

5.7. Decisões e aprendizagem

O desenvolvimento por etapas permitiu confirmar as ligações e testar as rotas antes de avançar para a integração. A separação das configurações da base de dados, a utilização de consultas preparadas, as transações e a validação dos dados foram decisões destinadas a tornar o código mais organizado, seguro e consistente. Os testes com dados de demonstração também ajudaram a identificar relações em falta entre encomendas e peças e a verificar os resultados das operações.

6. Testes realizados e resultados

6.1. Ligação PHP–MySQL

Foi utilizada uma página temporária teste.php para confirmar a ligação entre PHP e MySQL. O teste apresentou uma mensagem de ligação estabelecida com sucesso e indicou a existência de oito peças registadas na base de dados.

6.2. API de fornecedores

Foram testadas as operações de consulta, criação, atualização e desativação de fornecedores. Num teste de criação, a API devolveu o identificador 6 para o novo fornecedor. A desativação foi confirmada através da alteração do campo ativo, sem eliminar o registo.

6.3. Consulta e criação de encomendas

A consulta geral de encomendas devolveu os identificadores, fornecedores, datas, estados e valores totais. A consulta individual da encomenda com ID 1 apresentou o fornecedor e as duas peças associadas. A criação de encomendas foi testada através da API e da interface PHP. Num dos testes iniciais, a encomenda com ID 6, associada à Bosch Portugal, continha duas peças e apresentou um total de 87,10 €. Num teste posterior realizado através da interface, foi criada a encomenda número 8, com duas unidades de uma peça de teste a 10,50 € cada, totalizando 21,00 €.

A atualização do estado também foi testada. A encomenda com ID 6 passou de Pendente para Enviada. O envio do estado inválido Cancelada foi rejeitado com HTTP 400, e a tentativa de atualizar a encomenda inexistente ID 9999 devolveu HTTP 404.

6.4. Catálogo e filtros

O catálogo apresentou as peças registadas, incluindo referência, nome, marca, tipo, preço e stock. Foram testados os filtros por marca, tipo e intervalo de preços. Num dos testes, foram selecionados a marca Bosch, o tipo Filtros e um intervalo entre 10 € e 20 €, tendo os resultados correspondido aos critérios selecionados.

6.5. Registo de vendas e atualização do stock

Num teste de venda, foram registadas duas velas de ignição Bosch a 7,90 € cada, totalizando 15,80 €. O stock passou de 18 para 16 unidades. Num teste posterior, foi vendida a totalidade das 16 unidades restantes, e a base de dados confirmou que o stock passou para zero.

6.6. Integração e testes na interface

As páginas fornecedores.php e encomendas.php foram testadas no navegador e apresentaram os dados recebidos da API Node.js. A página encomendas_nova.php foi utilizada para criar uma encomenda e, posteriormente, a encomenda foi confirmada na listagem. Estes testes permitiram verificar os principais fluxos entre a interface PHP, a API Node.js e as duas bases de dados.

7. Dificuldades, limitações e melhorias futuras

7.1. Limitações identificadas

Foi identificada uma melhoria possível no catálogo: quando o preço mínimo é superior ao preço máximo, a página não apresenta uma mensagem específica que explique o problema. Poderá ser acrescentada uma validação explícita do intervalo.

Na API, a validação de preços poderá ser reforçada para limitar o número de casas decimais e impedir valores excessivamente elevados. Também poderá ser melhorado o tratamento de erros quando a aquisição de uma ligação à base de dados falha. As regras de integridade das tabelas poderão ser reforçadas para impedir quantidades negativas e preços inválidos.

A interface de vendas disponibiliza os métodos de pagamento definidos no código. A gestão desses métodos a partir da interface ou de uma tabela de configuração poderá ser considerada numa versão futura, caso se pretenda facilitar a introdução de novas opções.

7.2. Funcionalidades futuras

Uma melhoria proposta consiste em criar autenticação com dois perfis: administrador e vendedor. O administrador poderia gerir fornecedores e registar reposições de stock, enquanto o vendedor poderia consultar o catálogo, efetuar vendas e registar novos clientes. A implementação exigiria controlo de sessões e validação de permissões em cada operação protegida.

Outra funcionalidade futura seria uma página para consultar as vendas efetuadas num determinado dia, apresentando informações como cliente, peças, total e método de pagamento. Também seria útil permitir o registo de novos clientes diretamente pela interface PHP.

Na gestão de stock, poderia ser criado um histórico de entradas e saídas, permitindo saber quando e por que motivo uma quantidade foi alterada. Poderiam ainda ser acrescentados alertas para peças com stock reduzido e relatórios de vendas por período.

Por fim, será importante documentar e testar o processo de instalação num computador diferente, incluindo a criação das duas bases de dados, a instalação das dependências Node.js e a configuração das variáveis de ambiente.

8. Conclusão

O desenvolvimento da AutoLux permitiu construir uma aplicação web com catálogo de peças, filtros, registo de vendas e atualização de stock, bem como consulta e criação de encomendas a fornecedores através de uma API REST em Node.js. A utilização de duas bases de dados MySQL respeita a separação definida para os dados de vendas e para os dados de fornecedores.

O trabalho foi desenvolvido por etapas, com testes às ligações, às rotas e às páginas da interface. Os resultados obtidos confirmaram o funcionamento dos principais fluxos no ambiente local. O processo também permitiu aplicar conceitos de desenvolvimento web, bases de dados relacionais, APIs REST, validação de dados, transações e controlo de versões.

A aplicação pode ser ampliada com funcionalidades de autenticação, gestão de clientes, reposição de stock e consulta diária de vendas. Estas melhorias constituem oportunidades de evolução e não são apresentadas como funcionalidades já concluídas.